Assignment 06
Multimodal Search System for E-Commerce

Submission detail	Value
Student name	Nguyễn Gia Đức Trung
Student ID	B23DCVT423
Class	D23TTNT01

Abstract

This report presents a three-layer prototype for e-commerce product search using text, simulated voice, and image input. Each mode is normalized into a common query object. Candidate retrieval and result ranking are implemented as separate operations. The prototype searches a 12-product JSON dataset and a small in-memory image vector index; it also demonstrates a combined text-and-image query and customer-scoped order lookup. A 13-query Top-1 evaluation produced 12 successful results (92.31%). The remaining failure illustrates the limits of keyword matching when the customer uses synonyms. The report includes Visual Paradigm diagrams and program-output figures generated from the executable prototype.

Keywords

multimodal search; three-layer architecture; UML; keyword retrieval; image similarity; ranking

1. Introduction and Problem Description

1.1 Context

Online customers may express the same shopping need in different ways: typed keywords, a spoken sentence, or a picture of a desired product. A search system must convert these inputs into comparable representations, find candidate items, and present the most useful results. The assignment focuses on the system design and a working small prototype, rather than a production e-commerce platform.

1.2 Problem and objective

The problem is to support several input modes without creating a separate end-to-end architecture for each mode. The proposed solution uses one SearchUI, modality-specific conversion services, a common query dictionary, retrieval services, and a distinct ranking service. Products and orders are stored behind repositories. The user receives item names, prices, and ranking scores, or an order result when a known order ID is supplied.

1.3 Scope

- Required modes: English keyword search, voice search simulated from transcript text, and image-similarity search from a local PNG/JPEG file.
- Additional demonstrations: a weighted text-and-image query, a basic price ceiling, and order lookup constrained by customer ID.
- Dataset: 12 products, 12 product illustrations, two separate query illustrations, and three sample orders.
- Interface: a command-line SearchUI. No web server, microphone capture, or persistent database is required for this prototype.

1.4 Design principle

The processing path is: input → common query representation → candidate retrieval → ranking → results. The presentation layer collects input and displays output; the application layer interprets and searches; the data layer stores products, orders, and vectors. This layering prevents direct UI-to-database access and makes each search method replaceable.

2. Requirements Analysis

2.1 Actor, inputs, and outputs

The primary actor is the Customer. Inputs are (1) English keyword or natural-language text, (2) transcript text representing a voice request, (3) a local product image, and optionally (4) an order ID plus customer ID. Product-search output consists of ranked product records with product ID, name, price, and score. Order-search output consists of a matching order ID, status, total, and items, or an explicit not-found response.

2.2 Functional requirements

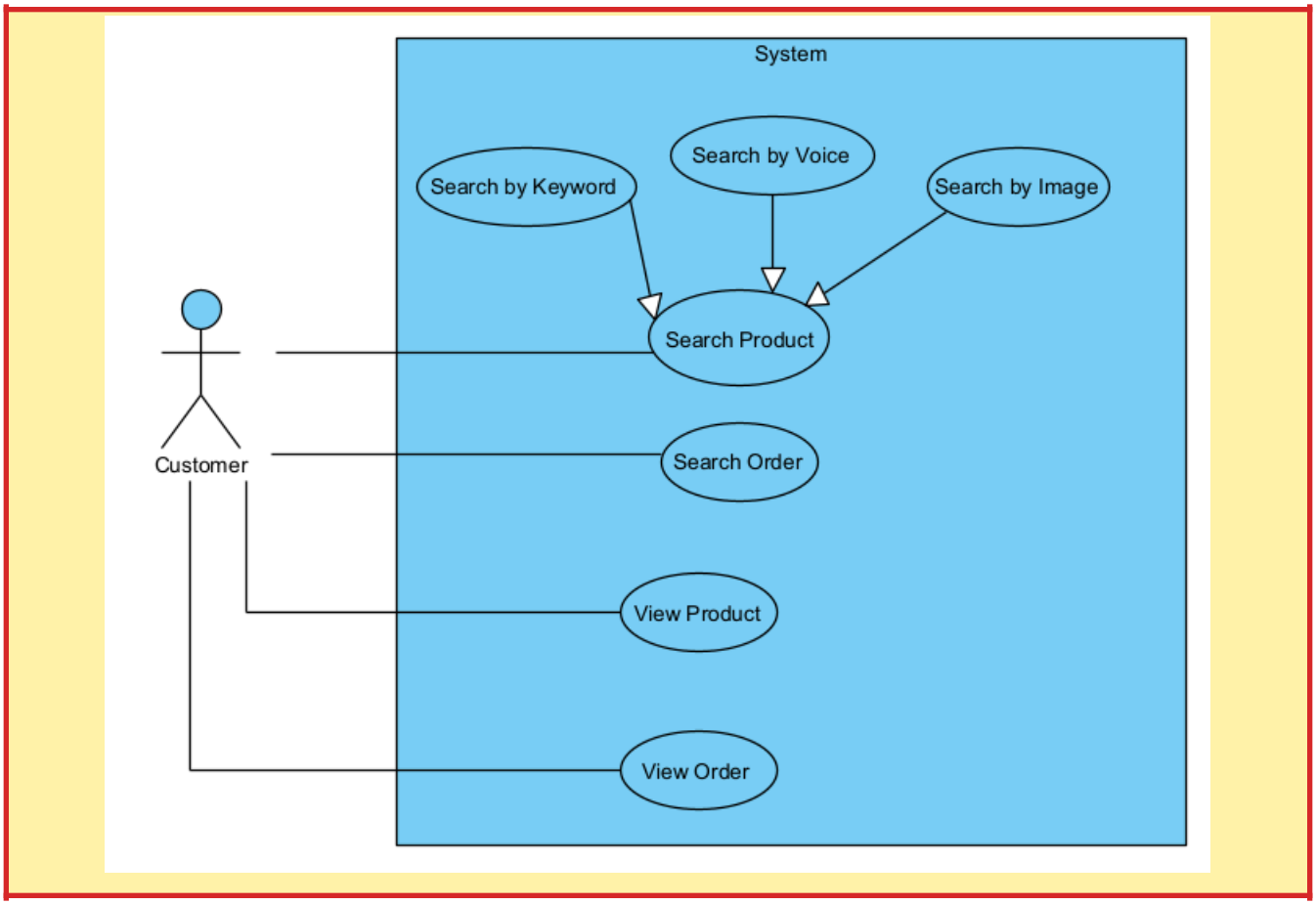
ID	Requirement	Implementation
FR1	Search products by typed text	Text query + keyword retrieval
FR2	Accept simulated voice input	SpeechService transcript + voice query
FR3	Search products by image	ImageService + cosine vector index
FR4	Use a common query format	QueryService dictionaries with type field
FR5	Rank and show product results	RankingService + SearchUI scores
FR6	View product information	Product records hold details; the CLI displays name and price (no separate detail command)
FR7	Search and view own order	OrderRepository checks customer ID
FR8	Combine text and image signals	Weighted multimodal ranking

2.3 Non-functional requirements

Maintainability: the system is divided into presentation, application, and data modules, and services have narrow responsibilities. Reproducibility: the sample data and illustration generator are included, with exact run commands in README.md. Usability: command output reports the input, processing path, candidate count, and scores. Reliability: missing image files and empty voice transcripts produce explicit errors. Privacy: a sample order is returned only when both its ID and customer ID match; this is a prototype constraint, not a replacement for authentication. Performance: the in-memory design is appropriate only for a small product set.

3. Use Case Model

The Customer may search for products using keyword, voice, or image input. These are three specialized forms of Search Product. The customer may then View Product. Separately, the customer may Search Order and View Order. The use cases state the customer's goals; the internal services that realize them appear in the component and sequence diagrams.



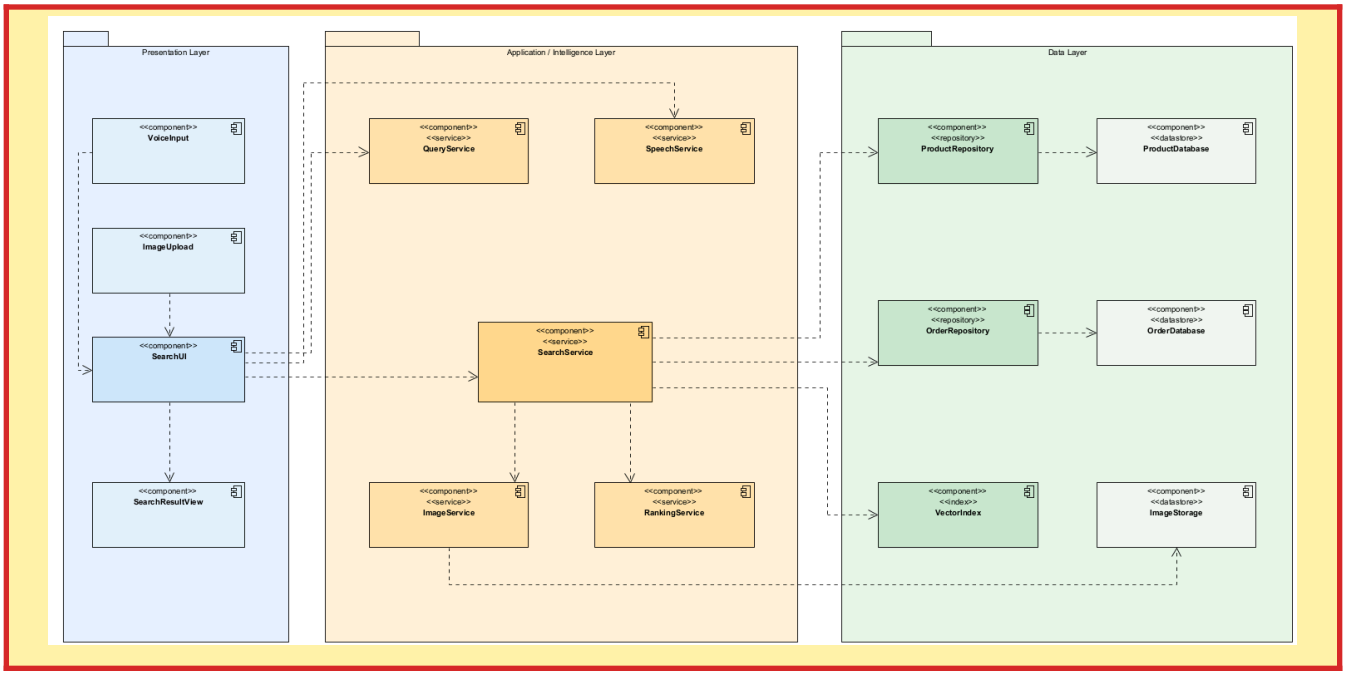
3.1 Use case descriptions

Use case	Precondition	Main outcome
Search by Keyword	Text entered	Ranked product list
Search by Voice	Transcript supplied	Transcript converted to text, then ranked products
Search by Image	Image file supplied	Visually similar ranked products
Search Order	Order ID and customer ID supplied	Matching order or not-found result

The voice branch uses a simulated transcript. In the UML model it may still be labeled voice search because the component boundary is the same as a future speech-to-text service. View Product and View Order are optional follow-up interactions after a search result is available.

4. Three-Layer Component Diagram with Packages

UML package	Components / modeled elements	Responsibility
Presentation Layer	SearchUI; VoiceInput; ImageUpload; SearchResultView	Collect mode-specific input and present output
Application / Intelligence Layer	QueryService; SpeechService; ImageService; SearchService; RankingService	Normalize, encode, retrieve, combine, and rank
Data Layer	ProductRepository; OrderRepository; VectorIndex; ProductDatabase; OrderDatabase; ImageStorage	Provide product, order, and vector data



4.1 Components and dependencies

The Component Diagram shows the components recommended by the handout inside three packages. The Presentation Layer contains SearchUI, VoiceInput, ImageUpload, and SearchResultView. The Application / Intelligence Layer contains QueryService, SpeechService, ImageService, SearchService, and RankingService. The Data Layer contains ProductRepository, OrderRepository, VectorIndex, ProductDatabase, OrderDatabase, and ImageStorage. The storage components represent JSON files and image files in the prototype.

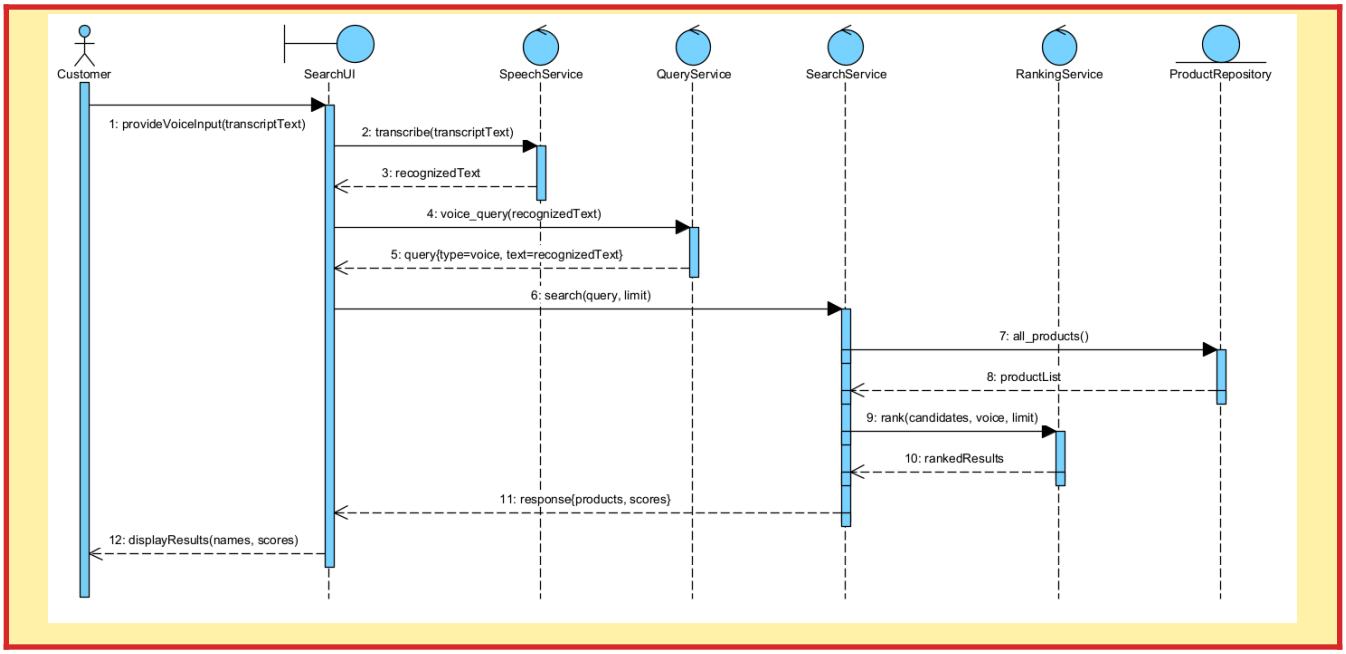
Dependencies follow Presentation -> Application / Intelligence -> Data. SearchUI uses QueryService, SpeechService, and SearchService; SearchService uses the repositories, VectorIndex, ImageService, and RankingService. SearchUI does not read JSON files or the vector index directly. ImageService computes image vectors, while ProductRepository and OrderRepository read their JSON files. The Python entry point wires these components together.

4.2 Retrieval versus ranking

Retrieval chooses a candidate set C(q) based on word overlap or cosine similarity. Ranking then orders C(q) using normalized relevance and a small stock signal. Keeping these steps separate makes the architecture visible in code and allows a later semantic retriever or business rule to be substituted without changing SearchUI.

5. Voice-Search Sequence Diagram

This interaction demonstrates the required sequence with Customer as a UML actor and six system lifelines: SearchUI, SpeechService, QueryService, SearchService, ProductRepository, and RankingService. SearchUI has the boundary stereotype; SpeechService, QueryService, SearchService, and RankingService have the control stereotype; ProductRepository has the entity stereotype. The Customer supplies transcript text as the simulated audio input. SearchUI asks SpeechService to transcribe it, creates a voice query via QueryService, and asks SearchService to retrieve and rank products.



Step	Message / result
1–3	Customer → SearchUI → SpeechService; recognized text returns
4–5	SearchUI → QueryService; normalized voice query returns
6–8	SearchUI → SearchService → ProductRepository; product list returns
9–10	SearchService → RankingService; ranked results return
11–12	SearchService → SearchUI → Customer; scored results displayed

The same application service receives a text query when the customer types a search. The image branch instead calls ImageService and VectorIndex. The voice sequence is therefore one concrete path through the shared architecture.

6. Python Implementation

6.1 Modules and data

Module	Purpose
presentation/search_ui.py	Input methods and result display
application/query_service.py	Common query dictionaries
application/speech_service.py	Simulated speech-to-text
application/image_service.py	Pixel-based visual vector
application/search_service.py	Candidate retrieval and modality routing
application/ranking_service.py	Final score and ordering
data/*_repository.py	Read product and order JSON data
data/vector_index.py	In-memory cosine candidate search

The product dataset contains 12 items across shoes, bags, clothing, electronics, and accessories. Each record stores an ID, name, brand, category, color, price, stock, description, and image path. The sample images were generated specifically for this prototype. Two additional query illustrations introduce slight visual changes so that the image demonstration is not a byte-for-byte lookup of a stored file.

Example product image and separate image query



Stored product: Red Travel Backpack



Separate query illustration: red backpack

6.2 Execution flow

The main.py entry point builds repositories and vectors, then routes the selected command to SearchUI. For a text or voice query, SearchService normalizes words and retrieves products with overlapping tokens. An English price phrase such as ‘under 100’ filters products before ranking. For an image query, ImageService reads the file, generates a vector, and VectorIndex returns products with the greatest cosine similarity. SearchUI prints the final score and component scores.

7. Multimodal Search Method

7.1 Common representation

QueryService produces a dictionary with a type field. Text and voice queries contain query text, image queries contain an image path, and combined queries contain both. This common envelope lets SearchService route the input without exposing the data layer to presentation details. Voice input first passes through SpeechService, which simply returns the provided transcript in this version.

7.2 Text and image signals

Text retrieval tokenizes lowercase English words, removes a small stop-word list, performs minimal singular normalization, and checks the product's name, brand, color, category, and description. Matches in the name, brand, or color receive more weight. A small category bonus makes a shoe search favor shoes over unrelated black products. The resulting text relevance St is capped at 1.

ImageService resizes each image to 96 × 96 pixels and extracts foreground color difference from white, a 12 × 12 spatial color grid, a 12 × 12 shape grid, mean foreground color, and foreground fraction. The descriptor is normalized to unit length. Cosine similarity is $Si(q,p) = (zq \cdot zp) / (||zq|| \, ||zp||)$. Because the vectors are normalized, VectorIndex can use their dot product to retrieve the closest products. This is a handcrafted image feature, not a learned image embedding.

7.3 Candidate retrieval and ranking

For text and voice, candidate products have at least one matching query term. For image, VectorIndex supplies the ten nearest products. Multimodal search merges the two candidate lists by product ID. RankingService then uses the following scores, with stock normalized to min(stock/20, 1):

- Text or voice: S = 0.98 St + 0.02 Sstock.
- Image: S = 0.98 Si + 0.02 Sstock.
- Text + image: S = 0.49 St + 0.49 Si + 0.02 Sstock.

The combined weights sum to 1. The equal text and image weights follow the assignment's suggested λt = λi = 0.5, while reserving a small share for an observable business signal. In the demonstration, ‘black running shoes’ and a separate black-shoe illustration jointly rank Nike Black Running Shoes first with score 0.9911.

8. Experimental Results

8.1 Evaluation setup

The evaluation uses 13 predefined queries. Success means the Top-1 product ID equals the expected ID; for the optional order lookup, the returned order ID must match. Query text, expected ID, actual result, score, and success are saved in results/evaluation.csv and results/evaluation.json. The demo transcript is in results/demo_results.txt. Image test queries use separate drawings from the stored product drawings.

Query	Mode	Expected	Top-1	Score	Success
black running shoes	text	1	1	0.992	Yes
red backpack	text	6	6	0.993	Yes
Nike shoes under 100 dollars	text	1	1	0.992	Yes
blue cotton t-shirt	text	7	7	1.0	Yes
find white sneakers	voice	2	2	0.995	Yes
find black headphones	voice	10	10	0.994	Yes
find brown leather watch	voice	11	11	0.987	Yes
red backpack query illustration	image	6	6	0.9928	Yes
black shoe query illustration	image	1	1	0.9902	Yes
black running shoes + black s…	multimodal	1	1	0.9911	Yes
red backpack + red backpack i…	multimodal	6	6	0.9929	Yes
footwear for jogging	text	1	—	—	No
order O001 for customer C001	order	O001	O001	N/A	Yes

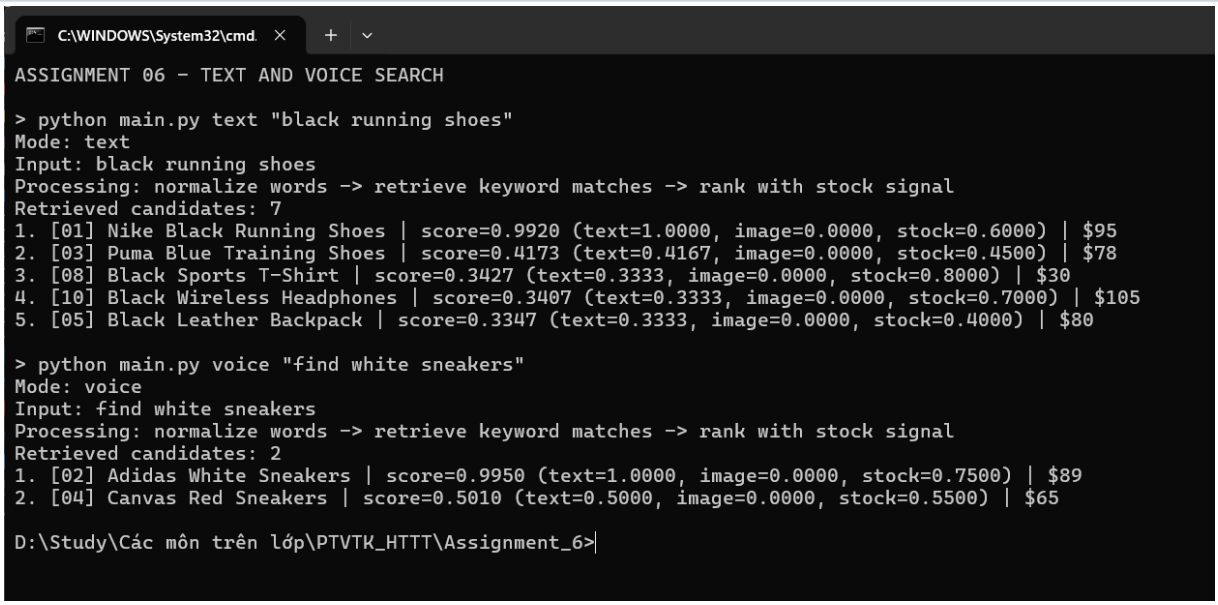
Successful queries: 12/13. Success rate = 92.31%. The failed query ‘footwear for jogging’ returned no result even though Nike Black Running Shoes was the predefined target. Neither ‘footwear’ nor ‘jogging’ appears as a searchable term in the product record; the current text method does not interpret those synonyms.

8.2 Demonstration of Required Search Modes

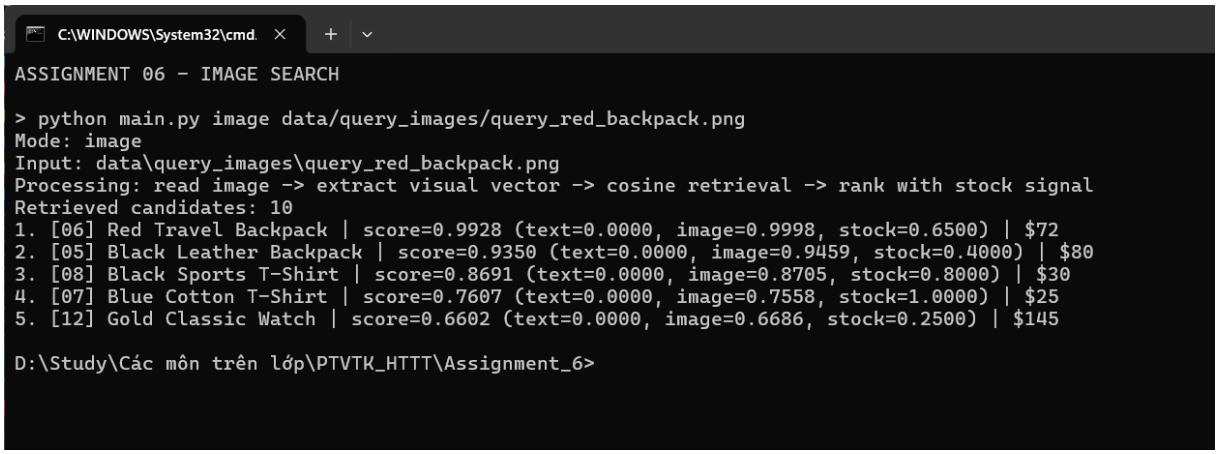
Each row below gives the actual input, processing path, returned products, and final ranking scores from the executable prototype. The voice input is a transcript supplied to the simulated SpeechService. The image input is a separate query illustration, not a stored product file.

Mode	Input	Processing	Returned products and final scores
Text	black running shoes	normalize words -> retrieve keyword matches -> rank with stock signal	1. Nike Black Running Shoes — 0.9920 2. Puma Blue Training Shoes — 0.4173 3. Black Sports T-Shirt — 0.3427
Voice (simulated)	find white sneakers	simulated speech-to-text → normalize words -> retrieve keyword matches -> rank with stock signal	1. Adidas White Sneakers — 0.9950 2. Canvas Red Sneakers — 0.5010
Image	data/query_images/query_red_backpack.png	read image -> extract visual vector -> cosine retrieval -> rank with stock signal	1. Red Travel Backpack — 0.9928 2. Black Leather Backpack — 0.9350 3. Black Sports T-Shirt — 0.8691

The full command-line transcript, including candidate counts and text/image/stock score components, is saved in results/demo_results.txt. The combined text-and-image demonstration is also present there and is explained in Section 7.



Screenshot: Text and simulated voice searches running in the terminal; input, processing, returned products, and scores are visible.

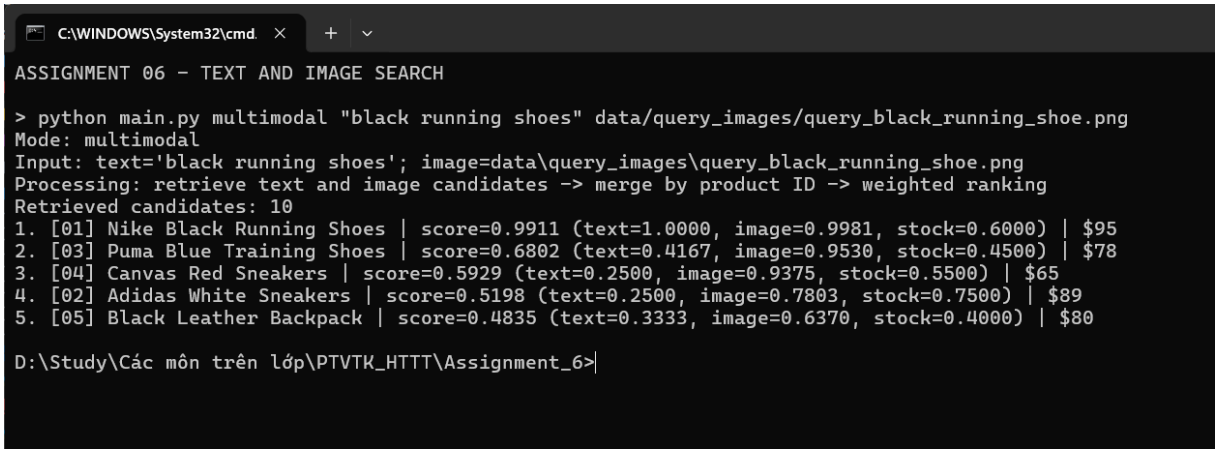


Screenshot: Image search running in the terminal with visual similarity and final ranking scores.

9. Discussion and Limitations

9.1 Interpretation

The results show that one application architecture can accept three input forms and return a consistently ranked product list. The separate query images ranked their intended products first, suggesting that the simple color-and-shape descriptor is adequate for the included illustrations. The experiment is small and controlled, so its 92.31% success rate should not be interpreted as expected performance on real customer photos or broad natural language.



Screenshot: Combined text-and-image search running in the terminal with component and final scores.

9.2 Known limitations and possible extensions

Speech recognition is simulated, so the system does not handle microphone audio or recognition errors. The handcrafted image descriptor is sensitive to backgrounds and changes in viewpoint. Keyword search misses semantic equivalents such as ‘jogging’ for ‘running.’ The CLI has no dedicated product-detail screen; full product fields remain in the repository. The JSON repository and linear vector scan are appropriate only for a tiny dataset. Stock is a deliberately small ranking signal, and the price parser recognizes only simple English phrases. A future version could add a real speech-to-text model, learned image/text embeddings, a scalable vector index, authenticated order access, and a web interface.

Order lookup in this classroom prototype checks the supplied customer ID but does not authenticate who supplied it. Production deployment would require account identity and authorization before returning order details. This matters because the Use Case Diagram includes order viewing, even though the principal assessment focus is product search.

10. Conclusion and Submission Map

The project satisfies the main technical requirements through a working three-layer Python prototype. Text, simulated voice, and image input are mapped to one query format; retrieval selects candidates and ranking orders them. The implementation includes 12 products, real pixel-based image descriptors, a weighted multimodal mode, order lookup, and saved demonstration results. The evaluation records both successful examples and a concrete failure. The supplied ass6.vpp contains the Use Case Diagram, three-layer Component Diagram, and voice-search Sequence Diagram. The report includes the three UML diagrams and the program-output figures for the required search modes.

10.1 Requirements traceability

Assignment task	Evidence in submission
Requirements analysis	Section 2 and functional requirements table
Use Case Diagram	Use Case Diagram in ass6.vpp and its figure in Section 3
Three-layer component diagram	Three-Layer Component Diagram in ass6.vpp and its figure in Section 4
Voice sequence diagram	Voice Search Sequence Diagram in ass6.vpp and its figure in Section 5
Python implementation	main.py, application/, presentation/, data/
Three-mode demonstration	Section 8.2, results/demo_results.txt, and three program-output figures
Experimental evaluation	evaluate.py, results/evaluation.csv/json

10.2 Files to submit

Submit the exported PDF report, the completed Visual Paradigm project (ass6.vpp), Python source code, data/products.json and data/orders.json, sample PNG images, README.md, and the results/ folder. Keep the editable DOCX for any final wording or layout edits. The submission also contains VISUAL_PARADIGM_GUIDE.md as a diagram checklist.

Reference

Course handout: Information System Analysis and Design, “Assignment 06: Multimodal Search System for E-Commerce,” 23-page PDF supplied with this assignment. All product data, sample illustrations, source code, and measured results in this report were created for this project.